

Distributional Rewards in MDPs

April 8, 2026

Abstract

Classical Markov Decision Processes (MDPs) suffer from exponential state-space growth in multi-agent systems. Recent work on distributional MDPs addresses this challenge by tracking agent distributions rather than individual states, successfully handling qualitative objectives such as safety and reachability. However, we are also interested in optimizing quantitative performance metrics. Traditional MDPs employ state-based reward functions, yet in multi-agent systems, meaningful rewards often depend on the collective distribution of agents rather than individual states.

We introduce distributional reward functions that assign rewards based on the collective agent distribution, enabling the expression of congestion penalties, formation quality metrics, load balancing objectives, and other aggregate performance measures. We develop a value iteration algorithm for computing optimal deterministic distributional policies under structured reward functions by representing value functions as maxima over finite vector sets. For the infinite-horizon setting, we establish existence and uniqueness of optimal value functions via contraction mapping arguments, proving geometric convergence of value iteration.

1. Introduction

Classical Markov Decision Processes (MDPs) have been successful for modeling sequential decision-making in single-agent systems. However, when we move to multi-agent settings, the classical state-based approach encounters a fundamental computational barrier: the state space grows exponentially with the number of agents. Consider m identical robots navigating an $n \times n$ grid—the state space has size n^{2m} , which becomes intractable even for modest swarm sizes. This exponential growth is not merely a computational inconvenience; it represents a fundamental mismatch between how we represent the system and what information we actually care about.

The Distributional Perspective. In many multi-agent sce-

narios, the identity of individual agents is irrelevant—what matters is the collective behavior. Do we really need to know that robot #47 is at position $(3, 5)$? Or do we only care that 15% of the robots are at position $(3, 5)$? This distributional perspective, introduced in (?) and developed further in (?), tracks the *distribution* of agents across states rather than individual agent positions. This distributional representation has dimension $|S|$ —completely independent of the number of agents—providing exponential compression. Moreover, distributional strategies can condition on global information (like “what fraction of agents have reached the goal?”) that individual agents cannot access.

Prior Work. The distributional perspective for MDPs, where system evolution is tracked as probability distributions over states rather than individual trajectories, was introduced in (?). These distributional properties are computationally hard—Skolem-hard for reachability (?)

Recent work has successfully synthesized (with semi-completeness guarantees) distributional strategies for *qualitative* specifications (??): safety objectives (maintaining distributions within safe sets) and reachability objectives (reaching target sets). The framework of (?) uses affine invariants and template-based synthesis, while (?) extends this to reach-avoidance properties with certified synthesis procedures.

Now that multi-agent systems can be modeled using the distributional view of MDPs, we also wish to optimize performance metrics, typically formulated as rewards. In this setting, traditional state-based rewards are insufficient, as they cannot capture many reward structures that arise naturally in multi-agent system as a state-based reward of the form $R(s, a, s')$ depends only on the local transition and does not depend on the global distribution at that time step.

We now present an example where state-based rewards cannot be used, at least not in a simple or natural manner, to capture the desired objective.

Example. Consider a warehouse floor consisting of storage cells, pathways, and packing stations. A large swarm of robots transports items to their final destinations. Our goal is to minimize the total time taken while also reducing congestion at each grid cell as the robots move.

We can model this system using the distributional view of MDPs. However, using state-based rewards fails to provide an appropriate formulation. Since a state-based reward assigns a value only to a single state transition, it cannot capture congestion effects. To penalize congestion (i.e., when more than an optimal number of robots occupy the same cell), we must know how many other robots are present in that cell, which is not available in the traditional reward framework.

In light of the above problems, In this paper we introduce a new type of reward function which depends on the entire global distribution which helps us formulate the reward functions arising in the multi-agent systems.

Contributions. Our main contributions are:

1. We introduce **distributional reward functions** $R_{\text{dist}} : \Delta(S) \times \Delta(A)^{|S|} \rightarrow \mathbb{R}$ that assign rewards based on the current distribution and action selections. This strictly generalizes classical rewards and enables expressing congestion penalties, formation quality metrics, and load balancing objectives.
2. We develop a **value iteration algorithm** for getting optimal deterministic policy (Section ??) for finite and infinite horizon cases.

1.1. Related Work

Distributional MDPs. The view of MDPs as distribution transformers, where the system state is represented as a probability distribution evolving over time, was introduced by (?). This perspective is particularly natural for modeling multi-agent systems, chemical reaction networks, and population protocols, where individual identities are irrelevant and only aggregate behavior matters. Early work by (?) introduced the iLTL logic for reasoning about probability mass functions in DTMCs, while (?) studied MDPs with unique compact invariant sets of distributions.

Computational Hardness. The verification and synthesis problems for distributional properties are computationally challenging. (?) showed that distributional reachability is as hard as the Skolem problem, a long-standing open problem in number theory whose decidability remains unknown (?). Furthermore, (?) demonstrated that distributional properties such as reachability and safety cannot be expressed in PCTL*, rendering classical probabilistic model checking techniques inapplicable.

Distributional Synthesis for Qualitative Objectives. Recent work has developed synthesis methods for qualitative distributional specifications. (?) introduced affine invariant synthesis for distributional safety objectives, showing that even simple MDPs may require infinite memory and randomization for distributional safety. This work was ex-

tended by (?) to handle distributional reach-avoidance properties, introducing the notion of distributional reach-avoid certificates and automated synthesis procedures. Both works employ template-based synthesis approaches (??) and provide soundness and relative completeness guarantees.

Interpretations of Probability. The chance-based and mass-based interpretations of probability provide two distinct philosophical frameworks for understanding probabilistic systems (?). The mass-based view, which describes distributions of probability mass across outcomes, aligns naturally with our distributional perspective on multi-agent systems.

Value Iteration for POMDPs. Our algorithmic approach draws inspiration from value iteration methods for Partially Observable MDPs (POMDPs) (??), which similarly operate over belief spaces (probability distributions over states). However, the distributional MDP setting presents distinct challenges due to the global nature of distributional rewards.

Classical MDPs with Rewards. Foundational work on MDPs with expected rewards (??) established the Bellman optimality equation and value iteration algorithms for finite state spaces. We extend these classical results to the distributional setting, proving existence and uniqueness of optimal value functions via contraction mapping arguments.

2. Preliminaries

Definition 1 (Markov Decision Process(MDP)). *An MDP is a tuple (S, A, T, γ) where S is a finite set of states, A is a finite set of actions, $T : S \times A \times S \rightarrow [0, 1]$ satisfies $\sum_{s'} T(s, a, s') = 1$, and $\gamma \in (0, 1)$ is the discount factor.*

$$\Delta(S) = \left\{ \mu : S \rightarrow [0, 1] \mid \sum_{s \in S} \mu(s) = 1 \right\}$$

$$\Delta(A) = \left\{ \mu : A \rightarrow [0, 1] \mid \sum_{a \in A} \mu(a) = 1 \right\}$$

State Based View of MDP : In this view the system evolves as $s_0, s_1, s_2, \dots$ where s_0 represents the initial state, s_t represents the state of system at time t .

Policies in State based view : At each timestep t , the system is in exactly one state s_t . The action taken at time t can depend on the entire past history of process, i.e. $(s_0, a_0, s_1, a_1, \dots, s_t)$. So in full generality, a policy is a function that maps the available history to a probability distribution over actions

$$\pi_t : H_T \rightarrow \Delta(A), \text{ where } H_T = (s_0, a_0, s_1, a_1, \dots, s_t)$$

A policy is called memory less if action at time step t just depends on s_t , $\pi : S \rightarrow \Delta(A)$. A policy is called deterministic if it takes exactly one action for each time step.

Another view to look at MDP is the **distributional view**

Which uses mass based of view of probability instead of chance based view of probability , (chance and mass based view of probability can be better understood from the paper (?)) .

Instead of tracking a single realized trajectory of states $(s_0, s_1, s_2, \dots)$, we can describe the system at each time step by a probability mass over states evolving over time . In the view, system evolves as $\mu_0, \mu_1, \mu_2, \dots$, where $\mu_t \in \Delta(S)$. Here μ_0 is the initial distribution and μ_t is distribution evolved at time t .

The Distributional Evolution happens as follows : Given distribution μ_t and policy $\pi_t : S \rightarrow \Delta(A)$, the next distribution is: $\mu_{t+1}(s') = \sum_{s \in S} \mu_t(s) \sum_{a \in A} \pi_t(s, a) T(s, a, s')$. Equivalently: $\mu_{t+1} = T_{\pi}^{\top} \mu_t$ where $[T_{\pi}]_{ss'} = \sum_a \pi_t(s, a) T(s, a, s')$.

Policies in this view can be similarly both history-dependent and memory-less .

General **History Dependent Policy** is as follows

$$\pi_t : H_t \rightarrow \Delta(A)^{|S|}$$

, $\pi_t(s, a)$ is probability of taking action a at state s , at time step t .

Note 1. We assume that at each time t , probability mass is distributed over all states and each portion of that mass takes actions according to the policy. This is different from the normal state-based view where the entire probability mass is in a single state. For example: $P_t(s_1 \text{ is observed at time } t) = P_t(\text{entire prob mass is in } s_1) = 0.4$ in the state-based view, in contrast to $\mu_t(s_1) = 0.4$ in the distributional view where 40% of the mass is at s_1 .

Similarly we can define memory-less policy as the one which just depends on present distribution to generate the action distributions $\pi : \mu \rightarrow \Delta(A)^{|S|}$. Similarly deterministic policy in this setting is taking exactly one action at each state , means $\exists a$ such that $p(s, a) = 1 \quad \forall s$

2.1. Value Functions and Bellman Operators

Value functions quantify the expected cumulative reward starting from a given state (or distribution) and following a particular policy. They are central to analyzing and solving MDPs.

Definition 2 (Value Function for State-Based MDPs). *Given a policy π and a reward function $R : S \times A \times S \rightarrow \mathbb{R}$, the value function $V^{\pi} : S \rightarrow \mathbb{R}$ is defined as:*

$$V^{\pi}(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \mid s_0 = s, \pi \right]$$

where the expectation is taken over the randomness in the policy and transitions.

The *optimal value function* $V^* : S \rightarrow \mathbb{R}$ is:

$$V^*(s) = \sup_{\pi} V^{\pi}(s)$$

The Bellman optimality equation states that V^* satisfies:

$$V^*(s) = \max_{a \in A} \left[\sum_{s' \in S} T(s, a, s') (R(s, a, s') + \gamma V^*(s')) \right]$$

Definition 3 (Bellman Operator). *The Bellman operator $\mathcal{T} : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$ is defined as:*

$$(\mathcal{T}V)(s) = \max_{a \in A} \left[\sum_{s' \in S} T(s, a, s') (R(s, a, s') + \gamma V(s')) \right]$$

The Bellman operator has several important properties:

Monotonicity. If $V_1(s) \leq V_2(s)$ for all $s \in S$, then $(\mathcal{T}V_1)(s) \leq (\mathcal{T}V_2)(s)$ for all $s \in S$.

Contraction. Under the supremum norm $\|V\|_{\infty} = \max_{s \in S} |V(s)|$, the Bellman operator is a γ -contraction:

$$\|\mathcal{T}V_1 - \mathcal{T}V_2\|_{\infty} \leq \gamma \|V_1 - V_2\|_{\infty}$$

By the Banach fixed point theorem, the contraction property implies that $\mathcal{T}$ has a unique fixed point, and this fixed point is precisely the optimal value function V^* . Moreover, the sequence $V_{k+1} = \mathcal{T}V_k$ converges to V^* for any initial V_0 , forming the basis of the value iteration algorithm.

Value Iteration Algorithm. Value iteration (??) is a fundamental dynamic programming algorithm for computing optimal policies in MDPs. Starting from an arbitrary initial value function V_0 (typically $V_0 = 0$), the algorithm iteratively applies the Bellman operator: $V_{k+1} = \mathcal{T}V_k$. Due to the contraction property, this sequence converges geometrically to the optimal value function V^* . Once V^* is obtained (or sufficiently approximated), the optimal policy can be extracted by choosing actions that achieve the maximum in the Bellman equation.

2.2. Lifting to the Distributional Setting

Our goal is to extend the value iteration framework to the distributional setting where we optimize over distributions $\mu \in \Delta(S)$ rather than individual states $s \in S$. This requires carefully defining distributional rewards and establishing analogous Bellman operators that preserve the key properties of monotonicity and contraction.

Definition 4 (Value Function for Distributional MDPs). *In the distributional setting with a distributional reward function $R_{\text{dist}} : \Delta(S) \times \Delta(A)^{|S|} \rightarrow \mathbb{R}$, the value function for a policy π is:*

$$V^{\pi}(\mu) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{\text{dist}}(\mu_t, \pi(\mu_t)) \mid \mu_0 = \mu, \pi \right]$$

where the distribution evolves according to $\mu_{t+1} = T_{\pi(\mu_t)}^\top \mu_t$.

The optimal value function in the distributional setting is:

$$V^*(\mu) = \sup_{\pi} V^{\pi}(\mu).$$

Note that $V^* : \Delta(S) \rightarrow \mathbb{R}$ maps from the infinite-dimensional probability simplex to the reals, in contrast to the finite-dimensional domain S in classical MDPs.

3. Distributional Rewards

We now formalize distributional reward functions and demonstrate their expressive power through concrete examples. Unlike traditional state-based rewards that depend only on local transitions, distributional rewards capture global system properties that emerge from collective agent behavior.

Definition 5 (Distributional Reward Function). A distributional reward function is a mapping

$$R_{\text{dist}} : \Delta(S) \times \Delta(A)^{|S|} \rightarrow \mathbb{R}$$

where $R_{\text{dist}}(\mu_t, \pi(\mu_t))$ represents the instantaneous reward at time t based on the current distribution μ_t and the action distribution $\pi(\mu_t)$ selected by the policy.

Definition 6 (Expected Cumulative Reward). For an initial distribution μ_0 and policy π , the expected cumulative reward is:

$$\mathbb{E}[R_{\mu_0}^{\pi}] = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{\text{dist}}(\mu_t, \pi(\mu_t)) \right]$$

where the expectation is taken over the stochastic evolution of distributions under policy π .

Remark 1 (Classical Rewards as Special Case). Any MDP with state-based reward $R : S \times A \times S \rightarrow \mathbb{R}$ admits an equivalent distributional reward formulation. Given R , define:

$$R_{\text{dist}}(\mu, \pi) = \sum_{s \in S} \mu(s) \sum_{a \in A} \pi(s, a) \sum_{s' \in S} T(s, a, s') R(s, a, s').$$

Then for any policy π and trajectory $(\mu_t)_{t \geq 0}$, we have $R_{\text{dist}}(\mu_t, \pi_t) = \mathbb{E}[R(s_t, a_t, s_{t+1}) \mid \mu_t, \pi_t]$, so the expected cumulative rewards coincide. Hence distributional rewards strictly generalize classical rewards; the converse fails since $R_{\text{dist}}(\mu, \pi)$ can depend nonlinearly on μ .

3.1. Motivating Example: Robot Swarm Navigation

Example 1 (Robot Swarm Navigation with Congestion). We consider a robot swarm moving on a grid with blocked

and destination regions (e.g., the IITB destination pattern used in our experiments). The swarm starts from a boundary distribution and must move toward the destination while avoiding high local occupancy.

The objective is to maximize a distributional reward that balances destination progress and congestion control. The reward model used in our implementation is:

$$R_{\text{dist}}(\mu) = \underbrace{w_{\text{move}} \cdot \min_{d \in D} \mu(d)}_{\text{destination progress term}} - \underbrace{\lambda \sum_{s \in S} \max\{0, \mu(s) - \kappa\}}_{\text{congestion overflow penalty}},$$

where D is the set of destination cells, κ is per-cell capacity, $w_{\text{move}} > 0$ corresponds to `move_cost`, and $\lambda > 0$ corresponds to `congestion_penalty`.

This formulation is distributional: each reward term depends on the global mass vector μ , not just a single transition (s, a, s') .

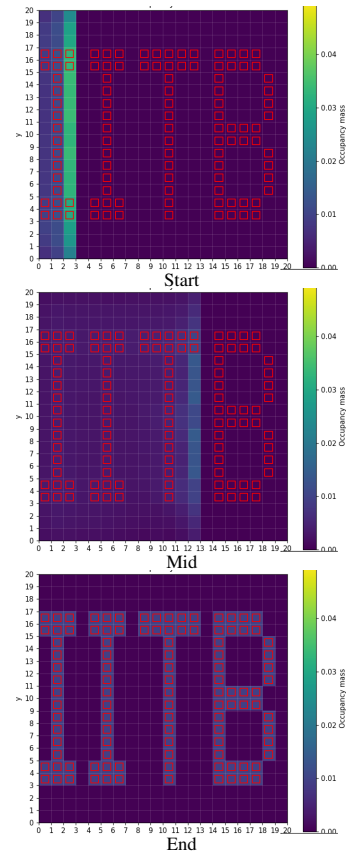


Figure 1. IITB destination example at start, intermediate, and end stages of policy rollout.

Example 2 (Warehouse Navigation with Path Concentration). Consider a 2×5 warehouse grid. Robots must transport items from start to destination, but the warehouse monitoring system can only effectively track robots when they move in concentrated groups rather than being dispersed across multiple paths.

Define two route corridors:

- $R_1 = \{s_{0,0}, s_{1,0}, s_{2,0}, s_{3,0}, s_{4,0}\}$ (bottom row)
- $R_2 = \{s_{0,1}, s_{1,1}, s_{2,1}, s_{3,1}, s_{4,1}\}$ (top row)

Objective: Transport all robots to destination while keeping them concentrated in a single corridor for effective monitoring.

Why Distributional Rewards: An individual robot cannot determine whether the swarm is concentrated or dispersed—this requires knowing the global distribution μ .

Reward Formulation: For each corridor R_i , let $\phi_i(\mu) = \sum_{s \in R_i} \mu(s)$ be the fraction of robots in that corridor. Define:

$$R_{\text{dist}}(\mu) = -c \cdot \sum_{s \notin S_{\text{dest}}} \mu(s) + \max_{i \in \{1,2\}} (-\lambda \cdot \phi_i(\mu))$$

where $S_{\text{dest}} = \{s_{4,0}, s_{4,1}\}$ are destination cells and $c, \lambda > 0$.

The first term penalizes robots not yet at destination (encouraging speed). The max-linear term penalizes the *less occupied* corridor: when robots split evenly with $\phi_1 = \phi_2 = 0.5$, we get $\max(-0.5\lambda, -0.5\lambda) = -0.5\lambda$. When concentrated with $\phi_1 = 0.9, \phi_2 = 0.1$, we get $\max(-0.9\lambda, -0.1\lambda) = -0.1\lambda$ (less penalty). An optimal policy concentrates robots in one corridor while minimizing travel time.

Example 3. (Reach Avoidance) We can also represent the safety constraints as mentioned in (?) as soft constraints (by adding huge negative penalty) using distributional rewards.

Say we are given an MDP planning problem with given safety constraints of the form $(\sum_s \mu(s) \cdot c_s^i \geq d^i \text{ for } i \in [1, N_c])$, then we can represent a reward function as

$$R_{\text{dist}}(\mu) = -c \cdot \sum_{s \notin S_{\text{dest}}} \mu(s) + \sum_{i \in N_c} P_i \cdot \min\left(0, \sum_s \mu(s) \cdot c_s^i - d^i\right)$$

Example 4 (Formation with Obstacle Avoidance). An army advances across a 10×6 grid from left to right. Two competing objectives drive the reward: formation cohesion (keep units concentrated in a tight block) and danger avoidance (stay away from hazardous regions). Both depend on the global distribution μ and cannot be expressed as state-based rewards.

Setup. Let $S = \{(i, j) : i \in \{1, \dots, 10\}, j \in \{1, \dots, 6\}\}$ be the 60-cell state space, with $\mu \in \Delta(S)$ the distribution of soldiers. For any subset $U \subseteq S$, write $\phi_U(\mu) = \sum_{s \in U} \mu(s)$ for the total mass in U .

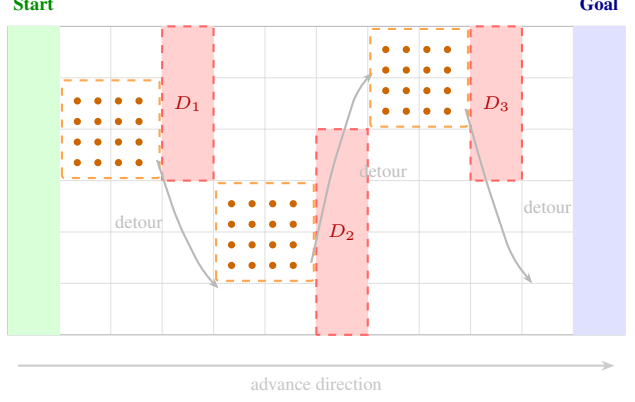


Figure 2. Army advances through a 12×6 grid with three staggered danger zones that force alternating detours.

Reward function. Fix a window size $r = 3$. Let $\mathcal{W}$ be the family of all $r \times r$ sliding windows:

$$\mathcal{W} = \{W_{ij} = \{(i', j') : i \leq i' < i+3, j \leq j' < j+3\}\},$$

so $|\mathcal{W}| = 8 \times 4 = 32$ windows in total. Let $D_1, D_2, D_3 \subset S$ be the three danger zones shown above, each with a safe-mass threshold $\kappa_\ell \in (0, 1)$. The reward is:

$$R_{\text{dist}}(\mu) = \underbrace{\alpha \mathbf{r}_{\text{prog}}^\top \mu}_{\text{progress}} + \underbrace{\beta \max_{W \in \mathcal{W}} \mathbf{1}_W^\top \mu}_{\text{formation}} - \underbrace{\sum_{\ell=1}^3 C_\ell \max(0, \phi_{D_\ell}(\mu) - \kappa_\ell)}_{\text{danger penalty}}, \quad (1)$$

where $\alpha, \beta, C_\ell > 0$ are tunable weights and each term captures a distinct aspect of the objective:

- **Progress** ($\alpha \mathbf{r}_{\text{prog}}^\top \mu$): The vector $\mathbf{r}_{\text{prog}} \in \mathbb{R}^{|S|}$ assigns $\mathbf{r}_{\text{prog}}(i, j) = i/10$ to each cell, so this term rewards mass that has advanced further to the right. It equals the (normalised) average column index of the army.
- **Formation** ($\beta \max_{W \in \mathcal{W}} \mathbf{1}_W^\top \mu$): $\mathbf{1}_W \in \{0, 1\}^{|S|}$ is the indicator of window W , so $\mathbf{1}_W^\top \mu = \phi_W(\mu)$ is the fraction of the army inside W . Maximising over all windows finds the tightest 3×3 block containing the most mass. This term equals 1 when the army is fully concentrated in some window and is small when dispersed.
- **Danger penalty** ($\sum_{\ell} C_\ell \max(0, \phi_{D_\ell}(\mu) - \kappa_\ell)$): If the mass inside danger zone D_ℓ exceeds threshold κ_ℓ , a penalty proportional to the excess is incurred. This forces the army to route around hazardous regions.

4. Optimality of Memoryless Distributional Policies

Before solving for optimal strategy for finding expected reward, we would like to know first what type of strategies

are sufficient and what are not.

In this Section we show 2 results

1. State-based memoryless policies are insufficient for optimizing expected reward over infinite horizon .
2. Distributional based memoryless policies are sufficient for optimizing expected reward over infinite horizon

Theorem 1. *State-based memoryless policies are not sufficient for optimizing expected reward under distributional reward functions.*

Proof. Consider states $S = \{s_1, s_2, s_3\}$, actions $A = \{a_1, a_2\}$, with transitions shown in Figure ??.

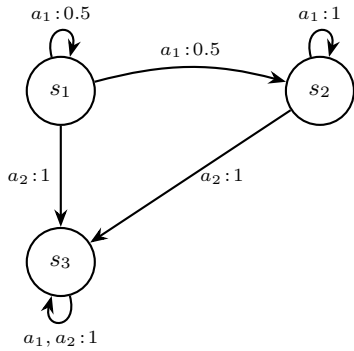


Figure 3. Counterexample MDP showing insufficiency of state-based policies.

Define $R_{\text{dist}}(\mu, \pi) = 1$ if $\mu = (0.5, 0.5, 0)$ and $\pi_{s_1}(a_2) = 1$, otherwise 0. With initial distribution $\mu_0 = (1, 0, 0)$:

Distributional strategy achieving positive reward: At $t = 0$: play a_1 at $s_1 \Rightarrow \mu_1 = (0.5, 0.5, 0)$. At $t = 1$: play a_2 at both s_1 and $s_2 \Rightarrow \mu_2 = (0, 0, 1)$. This yields expected reward $\gamma > 0$.

No state based memoryless policy achieves positive reward: To earn reward requires (1) reaching $\mu = (0.5, 0.5, 0)$ by playing a_1 at s_1 , and (2) playing a_2 at s_1 . But a state based memoryless policy uses the *same* action at s_1 at all times—it cannot play a_1 first then switch to a_2 . $\square$

Theorem 2. *Memoryless distributional policies are sufficient for optimizing expected reward over infinite horizon under distributional reward functions.*

Proof. Let π^* be an optimal policy (possibly history-dependent) generating trajectory $\tau = (\mu_0, \mu_1, \mu_2, \dots)$.

Case 1: All distributions in τ are distinct. Define $\tilde{\pi}(\mu_t)$ to be the action distribution that π^* selects at time t . Since each μ_t appears exactly once, $\tilde{\pi}$ is well-defined and achieves identical expected reward.

Case 2: Some distribution repeats. Suppose μ appears at times $t_1 < t_2$.

Claim 1. *If an optimal policy visits μ at two different times, there exists an optimal policy cycling through μ infinitely with identical intermediate trajectories.*

Let $R_{\text{cycle}} = \sum_{t=t_1}^{t_2-1} \gamma^{t-t_1} r_t$. Since π^* is optimal and returns to μ :

$$V^*(\mu) = R_{\text{cycle}} + \gamma^{t_2-t_1} \cdot V^*(\mu),$$

showing repeated cycles achieve optimal value.

From this claim, there exists an optimal policy whose trajectory is eventually periodic. Define $\tilde{\pi}(\nu)$ as the action distribution selected at distribution ν . This is well-defined since (a) before the cycle, each distribution appears at most once, and (b) within the cycle, we have a unique action for each distribution. This memoryless policy $\tilde{\pi}$ achieves optimal expected reward. $\square$

5. Hardness Results – New section

Theorem 3 (Policy evaluation is Skolem hard). *Given an MDP, a distributional reward function, and a policy π , deciding whether the expected total discounted reward over an infinite horizon is greater than 0 is Skolem-hard.*

Proof. We prove the result via reduction from a known Skolem-hard problem.

Given a Markov chain (S, T) , an initial state s_{start} , a state $s^* \in S$, and a threshold $c \in (0, 1)$, decide whether

$$\exists t \geq 0 \text{ such that } \mu_t(s^*) > c.$$

is Skolem-hard.

Given such a Markov chain, we construct an equivalent MDP by assigning a single action at every state. Thus, the MDP admits exactly one policy, and its dynamics coincide with the original Markov chain.

Now define a distributional reward function:

$$R_{\text{dist}}(\mu_t) = \max(0, \mu_t(s_i) - c).$$

Consider the total discounted reward:

$$\sum_{t=0}^{\infty} \gamma^t R_{\text{dist}}(\mu_t).$$

Observe:

- If there exists t such that $\mu_t(s_i) > c$, then $R_{\text{dist}}(\mu_t) > 0$ for that t , and hence the total reward is strictly positive.

- If for all t , $\mu_t(s_i) \leq c$, then $R_{\text{dist}}(\mu_t) = 0$ for all t , and the total reward is 0.

Therefore, deciding whether the total reward is greater than 0 is equivalent to deciding whether there exists t such that $\mu_t(s_i) > c$.

Since the latter problem is Skolem-hard, the result follows. $\square$

Theorem 4 (Optimality Checking is Skolem-hard). *Let $\mathcal{M} = (S, A, T, \gamma)$ be a finite MDP, $R_{\text{dist}} : \Delta(S) \times \Delta(A)^{|S|} \rightarrow \mathbb{R}$ a distributional reward function, and π is a distributional policy.*

Deciding whether π is optimal, i.e.,

$$V^\pi(\mu_0) = \sup_{\pi'} V^{\pi'}(\mu_0),$$

is Skolem-hard.

Proof. We reduce from the following Skolem-hard problem:

Problem. Given a Markov chain (S, T) , an initial state s_{start} , a state $s^* \in S$, and a threshold $c \in (0, 1)$, decide whether

$$\exists t \geq 0 \text{ such that } \mu_t(s^*) > c.$$

Construction. From the given Markov chain, construct an MDP $\mathcal{M}'$ as follows:

- State space: $S' = S \cup \{s_0, s_{\text{sink}}\}$.
- Actions:

$$A(s_0) = \{a_0, a_1\}, \quad A(s) = \{a_0\} \forall s \in S, \quad A(s_{\text{sink}}) = \{a_0\}.$$
- Transitions:
 - From s_0 :
 - * a_0 : transitions to s_{start} of markov chain
 - * a_1 : transitions to s_{sink}
 - For $s \in S$: follow the original Markov chain dynamics T
 - s_{sink} is absorbing

Reward function. Define the distributional reward:

$$R_{\text{dist}}(\mu) = \max(0, \mu(s^*) - c).$$

Policies. Consider the policy π defined by:

$$\pi(s_0) = a_1, \quad \pi(s) = a_0 \forall s \neq s_0.$$

This policy immediately transitions to the sink state and hence yields zero reward:

$$V^\pi(\mu_0) = 0.$$

Consider an alternative policy π' :

$$\pi'(s_0) = a_0, \quad \pi'(s) = a_0 \forall s \in S.$$

This policy simulates the original Markov chain.

Correctness of reduction.

- If $\exists t$ such that $\mu_t(s^*) > c$, then

$$R_{\text{dist}}(\mu_t) > 0,$$

and hence $V^{\pi'}(\mu_0) > 0$. Therefore π is *not* optimal.

- If for all t , $\mu_t(s^*) \leq c$, then

$$R_{\text{dist}}(\mu_t) = 0 \quad \forall t,$$

implying $V^{\pi''}(\mu_0) = 0 \quad \forall \pi''$. Hence π is optimal.

Thus, deciding whether π is optimal is equivalent to deciding whether

$$\exists t \text{ such that } \mu_t(s^*) > c,$$

which is Skolem-hard.

Therefore, optimality checking is Skolem-hard. $\square$

Theorem 5 (Optimal policy synthesis is skolem hard). *Given an MDP with a distributional reward function, computing an optimal policy for the infinite-horizon discounted objective is Skolem-hard.*

Proof. We reduce from the following Skolem-hard problem: given a Markov chain (S, T) , an initial state s_{start} , a state $s_i \in S$, and a threshold $c \in (0, 1)$, decide whether

$$\exists t \geq 0 \text{ such that } \mu_t(s_i) > c.$$

Construction. From the given Markov chain, construct an MDP $\mathcal{M}'$ as follows. The state space is

$$S' = S \cup \{s_0\}.$$

The action set is

$$A = \{a_0, a_1\}.$$

The available actions are:

$$A(s_0) = \{a_0\}, \quad A(s) = \{a_0, a_1\} \text{ for all } s \in S.$$

The transitions are defined by:

- from s_0 , action a_0 moves to the initial state $_{start}$ of the markov chain ;
- for every $s \in S$, action a_0 follows the original Markov chain transition matrix T ;
- for every $s \in S$, action a_1 sends the entire mass to s_0 .

Reward function. Define the distributional reward at time t by

$$R_{\text{dist}}(\mu_t) = -\mu_t(s_0) - M \cdot \max(\mu_t(s_i) - c, 0),$$

The total objective is the infinite-horizon discounted return

$$\sum_{t=0}^{\infty} \gamma^t R_{\text{dist}}(\mu_t).$$

Correctness of the reduction. Let π^0 denote the policy that always selects a_0 whenever a_1 is not chosen. Since a_0 is the only action at s_0 , this means that π^0 follows the original Markov chain and never uses a_1 .

Suppose that, under this trajectory, there exists a first time N such that

$$\mu_N(s_i) > c.$$

Then the policy π^0 incurs a penalty of at least

$$-M\gamma^N$$

from the term $-M \cdot \max(\mu_t(s_i) - c, 0)$, and the additional contribution from the term $-\mu_t(s_0)$ is nonpositive. Hence

$$V^{\pi^0}(\mu_0) \leq -M\gamma^N.$$

Now consider an alternative policy π^1 that uses a_1 at time $N-1$ and thereafter continues in the same manner whenever needed so as to prevent the distribution from reaching the threshold state s_i at time N . Under this policy, the penalty term $-M \cdot \max(\mu_t(s_i) - c, 0)$ is avoided at time N , and the only unavoidable cost is the term $-\mu_t(s_0)$. Since $\mu_t(s_0) \leq 1$ for every t , we obtain the lower bound

$$V^{\pi^1}(\mu_0) \geq -\sum_{t=N}^{\infty} \gamma^t = -\frac{\gamma^N}{1-\gamma}.$$

take

$$M > \frac{1}{(1-\gamma)},$$

we have

$$-\frac{\gamma^N}{1-\gamma} > -M\gamma^N.$$

Therefore,

$$V^{\pi^1}(\mu_0) > V^{\pi^0}(\mu_0),$$

so the policy that uses only a_0 is not optimal, the optimal must take action a_1 at some point.

On the other hand, if

$$\mu_t(s_i) \leq c \quad \text{for all } t,$$

then the penalty term is never activated. In that case, any use of a_1 only introduces extra visits to s_0 , and hence extra cost through the term $-\mu_t(s_0)$. Therefore, an optimal policy never uses a_1 .

Thus, an optimal policy uses a_1 if and only if the threshold event

$$\exists t \geq 0 \text{ such that } \mu_t(s_i) > c$$

occurs in the original Markov chain. Hence computing an optimal policy allows us to decide a Skolem-hard problem.

Therefore, optimal policy synthesis for this distributional-reward MDP is Skolem-hard. $\square$

Remark 2 (Intuition). *If the threshold is reached at time N , then the policy that never uses a_1 pays the large penalty $-M\gamma^N$. A policy that switches to a_1 before that point avoids this penalty and pays at most*

$$-\frac{\gamma^N}{1-\gamma},$$

which is strictly larger. This is exactly why the all- a_0 policy cannot be optimal when the threshold is reachable.

Conclusion. In the infinite-horizon setting, all problems involving distributional reward functions—namely policy evaluation, optimality verification, and optimal policy synthesis—are Skolem-hard.

Given that solving for infinite horizon is a hard problem even for simple reward functions as shown in ??, ??, ?? lets first try to solve the problem for finite horizon case

6. Convex Optimization Solution for concave reward functions in Finite-Horizon setting

For the finite horizon setting, if the distributional reward function is concave, the reward optimization problem can be written as convex optimization problem (maximizing over concave function)

Definition 7 (Occupancy Measure). *For each time step $t \in \{0, \dots, T-1\}$, define*

$$x_t(s, a) := \mu_t(s) \pi_t(a | s), \quad s \in S, a \in A.$$

Then $\mu_t(s) = \sum_{a \in A} x_t(s, a)$ for every $s \in S$.

Using these variables, the distributional dynamics become linear:

$$\mu_{t+1}(s') = \sum_{s \in S} \sum_{a \in A} x_t(s, a) T(s, a, s'), \\ \forall s' \in S, \forall t \in \{0, \dots, T-1\}.$$

We now formulate the finite-horizon problem directly in terms of $\{x_t, \mu_t\}$:

$$\begin{aligned} \max_{\{\mu_t, x_t\}} \quad & \sum_{t=0}^{T-1} \gamma^t R_{\text{dist}}(\mu_t, x_t) \\ \text{s.t.} \quad & \mu_0 = \mu_{\text{init}}, \\ & \sum_{a \in A} x_t(s, a) = \mu_t(s), \quad \forall s \in S, \forall t, \\ & \mu_{t+1}(s') = \sum_{s \in S} \sum_{a \in A} x_t(s, a) T(s, a, s'), \forall s' \in S, \forall t, \\ & x_t(s, a) \geq 0, \quad \forall s \in S, a \in A, \forall t. \end{aligned}$$

If the reward is affine in the occupancies, the problem reduces to a linear program.

Remark 3 (Policy Recovery). Given an optimal feasible solution $\{x_t^*, \mu_t^*\}$, an optimal policy is recovered by

$$\pi_t^*(a | s) = \begin{cases} \frac{x_t^*(s, a)}{\mu_t^*(s)}, & \mu_t^*(s) > 0, \\ \text{arbitrary in } \Delta(A), & \mu_t^*(s) = 0. \end{cases}$$

Note 2. The present solution will optimize over all possible policies, which can be time varying, the ?? only states that memoryless policies are sufficient in infinite horizon setting not in finite horizon

Remark 4. The examples ??,?? can solved using above approach for finite horizon setting, in fact for these examples can be directly formulated as an **Linear program** as piecewise-linear concave rewards of the form $-\max(0, \text{linear})$ or $\min(0, \text{linear})$ can be represented exactly using linear constraints and auxiliary variables

Slack Variable Reformulation. If a term of $-\max(x, y)$ is present in the reward function, create a variable z ,

$$z \geq x, y$$

, and now replace $-\max(x, y)$ term with $-z$.

7. Value Iteration solution for convex reward functions

The convex optimization approach of Section ?? handles concave distributional reward functions. We now consider

the complementary class: *convex* reward functions. Specifically, we focus on rewards that are **max-linear** in the distribution:

$$R_{\text{dist}}(\mu, \pi) = \max_{i \in [K]} \mathbf{a}_{\pi, i}^\top \mu,$$

where $\mathbf{a}_{\pi, i} \in \mathbb{R}^{|S|}$ are reward vectors indexed by the policy π and mode $i \in [K]$. Each term $\mathbf{a}_{\pi, i}^\top \mu$ is linear in μ , so the reward is a piecewise-linear convex function of μ for every fixed π .

Note 3. Any convex function can be approximated arbitrarily closely by a max-linear function (?), so this class is broadly expressive. In particular, the formation reward $\beta \max_{W \in \mathcal{W}} \mathbf{1}_W^\top \mu$ from Example ?? is directly max-linear with $K = |\mathcal{W}|$ and $\mathbf{a}_i = \beta \mathbf{1}_{W_i}$.

Restriction to deterministic policies : We optimize over the class of *deterministic* distributional policies.

Definition 8 (Deterministic Distributional Policy). A deterministic distributional policy at time t is a mapping $\pi_t : \Delta(S) \rightarrow A^S$ that assigns to each distribution $\mu \in \Delta(S)$ a single action $\pi_t(\mu)(s) \in A$ at every state s .

Let Π denote the (finite) set of all deterministic policies. Under any $\pi \in \Pi$, the distribution evolves *linearly*:

$$\mu'(s') = \sum_{s \in S} \mu(s) T(s, \pi(s), s') = (T_\pi^\top \mu)(s'),$$

where $[T_\pi]_{ss'} = T(s, \pi(s), s')$. This linearity is the key structural property that makes the value iteration algorithm tractable.

7.1. Finite-Horizon Value Iteration

Definition 9 (Optimal Value Function). For $t_0 \in \{0, \dots, T\}$ and $\mu \in \Delta(S)$, the optimal value function is:

$$V_{t_0}^*(\mu) = \sup_{\{\pi_t\}} \mathbb{E} \left[\sum_{t=t_0}^{T-1} \gamma^{t-t_0} R_{\text{dist}}(\mu_t, \pi_t(\mu_t)) \mid \mu_{t_0} = \mu \right],$$

where the supremum is over all sequences of deterministic policies $\{\pi_t\}_{t=t_0}^{T-1}$.

Proposition 6 (Bellman Recursion). For all $t_0 \in \{0, \dots, T-1\}$ and $\mu \in \Delta(S)$:

$$V_{t_0}^*(\mu) = \max_{\pi \in \Pi} [R_{\text{dist}}(\mu, \pi) + \gamma \cdot V_{t_0+1}^*(T_\pi^\top \mu)],$$

with boundary condition $V_T^*(\mu) = 0$ for all $\mu \in \Delta(S)$.

Proof. Separating the $t = t_0$ reward term and applying the principle of optimality to the remaining horizon gives:

$$V_{t_0}^*(\mu) = \max_{\pi \in \Pi} \left[R_{\text{dist}}(\mu, \pi) \right] \quad (2)$$

$$+ \gamma \cdot \sup_{\{\pi_t\}_{t>t_0}} \mathbb{E} \left[\sum_{t=t_0+1}^{T-1} \gamma^{t-t_0-1} R_{\text{dist}}(\mu_t, \pi_t) \right] \Big| \mu_{t_0+1} = \mu \quad (3)$$

The inner supremum equals $V_{t_0+1}^*(T_\pi^\top \mu)$ by definition. $\square$

Theorem 7 (Value Function is Max-Linear). *For all $t \in \{0, \dots, T\}$, the optimal value function is max-linear:*

$$V_t^*(\mu) = \max_{\mathbf{v} \in \Gamma_t} \mathbf{v}^\top \mu,$$

where $\Gamma_t \subset \mathbb{R}^{|S|}$ is a finite set of vectors.

Proof. We proceed by backward induction on t .

Base case ($t = T$). $V_T^*(\mu) = 0 = \mathbf{0}^\top \mu$ for all μ , so set $\Gamma_T = \{\mathbf{0}\}$.

Inductive step. Suppose $V_{t+1}^*(\mu) = \max_{\mathbf{v} \in \Gamma_{t+1}} \mathbf{v}^\top \mu$ for some finite set Γ_{t+1} . By the Bellman recursion (Proposition ??) and the max-linear reward:

$$\begin{aligned} V_t^*(\mu) &= \max_{\pi \in \Pi} \left[\max_{i \in [K]} \mathbf{a}_{\pi,i}^\top + \gamma \max_{\mathbf{v} \in \Gamma_{t+1}} \mathbf{v}^\top T_\pi^\top \mu \right] \\ &= \max_{\pi \in \Pi} \max_{i \in [K]} \max_{\mathbf{v} \in \Gamma_{t+1}} (\mathbf{a}_{\pi,i} + \gamma T_\pi \mathbf{v})^\top \mu, \end{aligned}$$

where the second equality uses the fact that i and $\mathbf{v}$ are independently optimized, so $\max_i x_i + \max_j y_j = \max_{i,j} (x_i + y_j)$.

Setting

$$\Gamma_t = \bigcup_{\pi \in \Pi} \bigcup_{i \in [K]} \left\{ \mathbf{a}_{\pi,i} + \gamma T_\pi \mathbf{v} : \mathbf{v} \in \Gamma_{t+1} \right\}$$

gives $V_t^*(\mu) = \max_{\mathbf{w} \in \Gamma_t} \mathbf{w}^\top \mu$. Since $|\Pi|$, K , and $|\Gamma_{t+1}|$ are all finite, so is $|\Gamma_t|$. $\square$

Algorithm ?? implements this backward recursion directly.

[t] Finite-Horizon Value Iteration for Max-Linear Distributional Rewards [1] **Input:** Horizon T , discount γ , transition matrices $\{T_\pi\}_{\pi \in \Pi}$, reward vectors $\{\mathbf{a}_{\pi,i}\}_{\pi,i}$ **Initialize:** $\Gamma_T \leftarrow \{\mathbf{0}\}$ $t = T - 1, T - 2, \dots, 0$ $\Gamma_t \leftarrow \emptyset$ each policy $\pi \in \Pi$ each reward mode $i \in [K]$ each vector $\mathbf{v} \in \Gamma_{t+1}$ $\Gamma_t \leftarrow \Gamma_t \cup \{\mathbf{a}_{\pi,i} + \gamma T_\pi \mathbf{v}\}$ **(Optional)** Remove dominated vectors from Γ_t **Output:** $\{\Gamma_t\}_{t=0}^T$

Given the output $\{\Gamma_t\}$, the optimal action at time t from distribution μ is recovered by:

$$\pi_t^*(\mu) = \arg \max_{\pi \in \Pi} \max_{i \in [K]} \max_{\mathbf{v} \in \Gamma_{t+1}} (\mathbf{a}_{\pi,i} + \gamma T_\pi \mathbf{v})^\top \mu.$$

Remark 5 (Set size and pruning). *The set Γ_t can grow as $O((|\Pi| \cdot K)^{T-t})$ in the worst case. The optional pruning step removes a vector $\mathbf{w} \in \Gamma_t$ if there exists $\mathbf{w}' \in \Gamma_t$ with $\mathbf{w}'^\top \mu \geq \mathbf{w}^\top \mu$ for all $\mu \in \Delta(S)$. This does not affect correctness and can reduce the set size in practice.*

Remark 6. *Similar methodology is used in (2) where the authors introduce rewards on belief states in PoMDP's and use similar value iteration algorithm of PoMDP's*

8. Infinite Horizon Case – To be done

Theorem 8 (Existence via Contraction Mapping). *For discount factor $\gamma < 1$, there exists a unique optimal value function $V^* : \Delta(S) \rightarrow \mathbb{R}$ satisfying:*

$$V^*(\mu) = \max_{\pi \in \Pi} [A_\pi^\top \mu + \gamma \cdot V^*(T_\pi^\top \mu)].$$

Proof. Let $\mathcal{B}(\Delta(S))$ be bounded continuous functions on $\Delta(S)$ with supremum norm $\|V\|_\infty = \sup_\mu |V(\mu)|$ —a complete metric space.

Define Bellman operator $\mathcal{T} : \mathcal{B}(\Delta(S)) \rightarrow \mathcal{B}(\Delta(S))$:

$$(\mathcal{T}V)(\mu) = \max_{\pi \in \Pi} [A_\pi^\top \mu + \gamma \cdot V(T_\pi^\top \mu)].$$

Contraction property: For any V_1, V_2 and μ :

$$|(\mathcal{T}V_1)(\mu) - (\mathcal{T}V_2)(\mu)| \leq \max_\pi \gamma |V_1(T_\pi^\top \mu) - V_2(T_\pi^\top \mu)| \leq \gamma \|V_1 - V_2\|_\infty$$

Thus $\|\mathcal{T}V_1 - \mathcal{T}V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$. By the Banach Fixed Point Theorem, a unique $V^* = \mathcal{T}V^*$ exists. $\square$

In the following section we reverse the time direction in value iteration algorithm

Theorem 9 (Convergence of Value Iteration). *Starting from any $V_0 \in \mathcal{B}(\Delta(S))$, the sequence $V_{k+1} = \mathcal{T}V_k$ converges uniformly:*

$$\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty \rightarrow 0.$$

Proof. Since $\mathcal{T}$ is a contraction with modulus γ : $\|V_{k+1} - V^*\|_\infty = \|\mathcal{T}V_k - \mathcal{T}V^*\|_\infty \leq \gamma \|V_k - V^*\|_\infty$. By induction, $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$, giving geometric convergence. $\square$

Algorithm for Infinite Horizon. The algorithm for the infinite horizon case follows the same structure as the finite horizon case but iterates until convergence. Starting with $\Gamma_0 = \{\mathbf{0}\}$, we repeatedly compute:

$$\Gamma_{k+1} = \bigcup_{\pi \in \Pi} \{A_\pi + \gamma \cdot T_\pi \gamma_k : \gamma_k \in \Gamma_k\}$$

and prune dominated vectors. The iteration continues until $\|V_{k+1} - V_k\|_\infty < \epsilon$ for some convergence threshold $\epsilon > 0$,

where $V_k(\mu) = \max_{\gamma \in \Gamma_k} \gamma^\top \mu$. By standard arguments using the contraction property, if $\|V_{k+1} - V_k\|_\infty < \epsilon$, then $\|V_{k+1} - V^*\|_\infty \leq \frac{\gamma\epsilon}{1-\gamma}$. Theorem ?? guarantees geometric convergence of this procedure.

8.1. Computing the Supremum Norm

To implement value iteration, we need to compute the stopping criterion $\|V_{k+1} - V_k\|_\infty = \sup_{\mu \in \Delta(S)} |V_{k+1}(\mu) - V_k(\mu)|$. Since $V_k(\mu) = \max_{\gamma \in \Gamma_k} \gamma^\top \mu$ is piecewise linear and convex, this supremum is attained at a vertex of $\Delta(S)$.

Lemma 10 (Supremum Computation). *For piecewise linear value functions $V_k(\mu) = \max_{\gamma \in \Gamma_k} \gamma^\top \mu$, the supremum norm can be computed as:*

$$\|V_{k+1} - V_k\|_\infty = \max_{s \in S} |V_{k+1}(e_s) - V_k(e_s)|$$

where $e_s \in \Delta(S)$ is the point mass at state s (i.e., $e_s(s) = 1$ and $e_s(s') = 0$ for $s' \neq s$).

Proof. Since V_k is the maximum of linear functions, it is convex and piecewise linear. For convex functions over the simplex $\Delta(S)$, the maximum is attained at a vertex. The vertices of $\Delta(S)$ are precisely the point masses $\{e_s : s \in S\}$. Therefore:

$$\begin{aligned} \|V_{k+1} - V_k\|_\infty &= \sup_{\mu \in \Delta(S)} |V_{k+1}(\mu) - V_k(\mu)| \\ &= \max_{s \in S} |V_{k+1}(e_s) - V_k(e_s)| \\ &= \max_{s \in S} \left| \max_{\gamma \in \Gamma_{k+1}} \gamma(s) - \max_{\gamma \in \Gamma_k} \gamma(s) \right| \end{aligned}$$

This can be computed in $O(|S| \cdot (|\Gamma_k| + |\Gamma_{k+1}|))$ time. $\square$

9. Experiments :

For concave reward function case in ??:

Table 1. Experiment runtime summary.

Shape	Grid Size	Is Randomized	Time Taken (s)
iitb	20x20	No	30.513
smiley	20x20	No	29.260
iitb	40x40	No	350.202
smiley	40x40	No	353.572
iitb	20x20	Yes	201.793
iitb	20x20	Yes	1000.142 (timeout)
smiley	20x20	Yes	1000.167 (timeout)
smiley	20x20	Yes	1000.174 (timeout)
iitb	40x40	Yes	1000.295 (timeout)
iitb	40x40	Yes	1000.211 (timeout)
smiley	40x40	Yes	1000.248 (timeout)
smiley	40x40	Yes	1000.301 (timeout)